In C++, the object itself is the default.

A pointer needs a reason.

```
Player a;                    // the default
Player* b = new Player;      // needs a reason
```

Coming from Java or C#, it feels backwards.

swipe →

What a pointer costs

	`PLAYER A;`	`NEW PLAYER`
Freed	automatically, at the `}`	**by you**, with `delete`
Where	the stack	the heap
Access	direct	one extra hop

Forget the `delete` and it leaks. Delete twice and it crashes.

When you really need one

YOU NEED	BECAUSE
polymorphism	a `Base*` can point to any `Derived`
to outlive the scope	the object must survive the `}`
"no object"	`nullptr` means absent
to share one object	several owners, one instance

Big objects are not a reason by themselves: a `std::vector` already keeps its data on the heap.

Even then, skip the raw pointer

YOU NEED	USE
polymorphism, one owner	`std::unique_ptr<Base>`
shared ownership	`std::shared_ptr<T>`
maybe a value	`std::optional<T>`
just refer to one	`T&`

They free themselves, so there is no `delete` to forget.

The classic bug

```
Player* make() {
    Player p;
    return &p;        // p dies at the }
}

Player make() {
    return Player();  // fine
}
```

The first returns a dangling pointer.

Returning the object by value is safe, and the compiler usually builds it straight in the caller, with no copy.

The order to try

#	REACH FOR
1	the object itself
2	a reference
3	a smart pointer
4	**a raw `new`**, last

How many raw `new` are in your code?